

`Math.PI` is a static member variable

To access a private static class member when the class is not instantiated, you still prefix with the class name, but then you also use a public static method:

```
SavingsAccount.getInterestRate()
```

When even a single instance of the class `SavingsAccount` exists (is instantiated), then any of those existing classes can access our static variable (`interestRate`) simply by using its name: `interestRate`. When no objects of class `SavingsAccount` exist, our static variable can still be referenced, but only by going through the Class name and a public static method:

```
SavingsAccount.getInterestRate()
```

Just like static class members (data variables), belong to the class not any one instantiation of the class. Static class methods do not operate on any instance of a class. That means you can use them without creating any instance of a class. For example, all the methods built into the `Math` class are static methods. You use this general syntax when using :

```
ClassName.staticMethod( parameters );
```

Because static methods do not work with an instance of a class, they can only access static fields. Let's think about this: No instance of the class is instantiated. In this situation, the only thing that could possibly be present is a static member. Thus, the only thing that would be around for a static method to see is another static member. Finally, consider the most famous of all static methods:

```
public static void main( String args[] )
```

Since `main` is static, you don't need to create an instance of the class in order to call it—and the Java interpreter doesn't either. Once again, “static” means: variables and methods that belong to a class but not to any particular object of the class. 